Séance 03 - Conditions et logique booléenne en PHP

Table of Contents

1. Situation professionnelle . . . 2
2. Objectifs . . . 2
3. Prérequis . . . 2
4. Réactivation - Variables et opérateurs . . . 2
5. Activité 1 - Préparer la séance . . . 3
   5.1. Vérifier le dépôt . . . 3
   5.2. Démarrer le serveur PHP . . . 3
   5.3. Où vont vos fichiers . . . 3
6. Activité 2 - Produire des valeurs booléennes . . . 4
   6.1. Les opérateurs de comparaison . . . 4
   6.2. Observer des comparaisons . . . 4
   6.3. Comprendre l'égalité stricte . . . 5
7. Activité 3 - Exécuter une instruction sous condition . . . 5
   7.1. Utiliser `if` . . . 5
   7.2. Utiliser `if/else` . . . 6
   7.3. Premier commit de la séance . . . 6
8. Activité 4 - Classer une mesure avec `elseif` . . . 6
9. Activité 5 - Combiner plusieurs conditions . . . 7
   9.1. Opérateurs logiques . . . 7
   9.2. Tester la disponibilité d'un service . . . 7
   9.3. Déclencher une alerte sur plusieurs ressources . . . 8
10. Activité 6 - Déterminer l'état global d'un serveur . . . 9
11. Activité 7 - Corriger des erreurs logiques . . . 9
12. Exercice autonome - Autoriser une mise à jour . . . 10
13. Travail sur papier . . . 11
14. Exercices bonus . . . 11
   14.1. Exercice bonus 1 [*] - Fenêtre de maintenance . . . 11
   14.2. Exercice bonus 2 [**] - Éligibilité à une sauvegarde . . . 12
   14.3. Exercice bonus 3 [*] - Défi : prévoir la sortie . . . 12
15. Synthèse . . . 13
16. Validation Git . . . 13
17. Arrêter le serveur PHP . . . 14

1. Situation professionnelle

L'application de supervision doit interpréter les mesures reçues depuis les serveurs.

Afficher une valeur ne suffit plus. Le programme doit maintenant prendre des décisions : signaler un serveur arrêté, détecter une charge anormale, distinguer un état normal d'un état critique ou autoriser une opération seulement lorsque plusieurs conditions sont réunies.

Dans cette séance, les décisions seront prises à partir de valeurs écrites directement dans les programmes. Les formulaires, les tableaux et les boucles seront étudiés plus tard.

2. Objectifs

À la fin de la séance, vous devrez être capable de :

- construire une expression booléenne ;
- utiliser les opérateurs de comparaison ;
- distinguer affectation et comparaison ;
- écrire une structure `if`, `if/else` et `if/elseif/else` ;
- combiner plusieurs conditions avec `&&`, `||` et `!` ;
- choisir l'ordre correct des tests ;
- prévoir le résultat d'un programme sans l'exécuter ;
- repérer et corriger une erreur logique ;
- versionner le travail avec Git.

3. Prérequis

- La séance 01 sur Git est terminée.
- La séance 02 sur les variables, les types et les opérateurs est terminée.
- Vous avez le dossier du TP (avec `compose.yml`, `index.php` et `diagnostic.php`).
- Docker Desktop est démarré (icône baleine).



Rappel du démarrage : ouvrez le dossier du TP, tapez `docker compose up -d`, puis ouvrez `http://localhost:8181`. Vos fichiers `.php` vont **dans le dossier du TP**. Guide pas à pas : `cours/annexe-docker-php.adoc`.

4. Réactivation - Variables et opérateurs

Sans utiliser l'ordinateur, répondez sur papier.

1. Quel est le type de la valeur `true` ?

2. Quel opérateur permet de concaténer deux chaînes en PHP ?
3. Quelle est la valeur de `$resultat` ?

```
$prix = 840;
$duree = 4;
$resultat = $prix / $duree;
```

4. Quelle fonction affiche à la fois la valeur et le type d'une variable ?
5. Quelle différence existe entre `$estActif = true;` et `echo $estActif;` ?

5. Activité 1 - Préparer la séance

5.1. Vérifier le dépôt

Ouvrez la racine de votre dépôt dans Visual Studio Code.

Dans le terminal du projet :

```
git status
git pull
```

Le dépôt doit être propre avant de commencer.

5.2. Démarrer le serveur PHP

Dans le terminal ouvert dans le dossier du TP, démarrez le serveur (s'il ne tourne pas déjà) :

```
docker compose up -d
```

Vérifiez `http://localhost:8181/` : la page verte doit s'afficher.

5.3. Où vont vos fichiers

Vos fichiers `.php` se créent **dans le dossier du TP** (à côté de `compose.yml`). Le fichier `diagnostic.php` y est déjà. Ils s'ouvrent avec des URL de la forme :

```
http://localhost:8181/nom-du-fichier.php
```

6. Activité 2 - Produire des valeurs booléennes

Une condition est une expression dont le résultat est soit `true`, soit `false`.

6.1. Les opérateurs de comparaison

Opérateur	Signification	Exemple
`===`	Strictement égal à : même valeur et même type	`$etat === "actif"`
`!==`	Strictement différent de	`$etat !== "actif"`
`>`	Strictement supérieur à	`$chargeCpu > 80`
`>=`	Supérieur ou égal à	`$chargeCpu >= 80`
`<`	Strictement inférieur à	`$memoireLibreGo < 2`
`⇐`	Inférieur ou égal à	`$espaceLibreGo ⇐ 10`



`=` affecte une valeur à une variable. `===` compare deux valeurs. Les confondre produit une erreur logique difficile à repérer.

6.2. Observer des comparaisons

Créez `comparaisons.php` :

```
<?php

$chargeCpu = 75;
$seuilAlerte = 70;
$etatServeur = "actif";
$nombreServices = 4;

var_dump($chargeCpu >= $seuilAlerte);
var_dump($chargeCpu < 50);
var_dump($etatServeur === "actif");
var_dump($nombreServices !== 0);
```

Avant d'exécuter le fichier, prévoyez les quatre résultats.

Exécutez ensuite le programme dans le navigateur et comparez vos réponses avec les résultats obtenus.

6.3. Comprendre l'égalité stricte

Ajoutez temporairement :

```
$codeEtat = "1";

var_dump($codeEtat == 1);
var_dump($codeEtat === 1);
```

Répondez aux questions suivantes :

1. Les deux comparaisons donnent-elles le même résultat ?
2. Quelle comparaison vérifie également le type ?
3. Quel opérateur d'égalité utiliserons-nous par défaut dans le module ?



L'opérateur `==` autorise des conversions automatiques. Pour rendre les intentions du programme plus claires, nous privilégierons `===` et `!==`.

7. Activité 3 - Exécuter une instruction sous condition

7.1. Utiliser `if`

Créez `alerte-disque.php` :

```
<?php

$nomServeur = "srv-web-01";
$espaceLibreGo = 8;
$seuilAlerteGo = 10;

if ($espaceLibreGo <= $seuilAlerteGo) {
    echo "ALERTE : espace disque insuffisant sur " . $nomServeur . ".";
}
```

Exécutez le programme avec les valeurs suivantes :

Espace libre	Seuil	Résultat attendu
8	10	L'alerte est affichée.
10	10	L'alerte est affichée.
15	10	Aucun texte n'est affiché.

QUESTION: Pourquoi la valeur `10` déclenche-t-elle l'alerte ?

7.2. Utiliser `if/else`

Créez `etat-serveur.php` :

```
<?php

$nomServeur = "srv-web-01";
$estActif = true;

if ($estActif) {
    echo $nomServeur . " est actif.";
} else {
    echo $nomServeur . " est arrêté.";
}
```

Testez le programme une fois avec `true`, puis une fois avec `false`.

QUESTION: Pourquoi écrit-on simplement `if ($estActif)` plutôt que `if ($estActif === true)` ?

7.3. Premier commit de la séance

Depuis la racine du dépôt :

```
git status
git add 03-conditions/comparaisons.php 03-conditions/alerte-disque.php 03-
conditions/etat-serveur.php
git commit -m "Ajoute les premières conditions PHP"
git push
```

8. Activité 4 - Classer une mesure avec `elseif`

Un serveur est classé selon sa charge processeur :

- à partir de 90 %, l'état est `CRITIQUE` ;
- à partir de 70 %, l'état est `SURVEILLANCE` ;
- en dessous de 70 %, l'état est `NORMAL`.

Créez `charge-cpu.php` :

```
<?php

$chargeCpu = 75;
```

```
if ($chargeCpu >= 90) {
    $etat = "CRITIQUE";
} elseif ($chargeCpu >= 70) {
    $etat = "SURVEILLANCE";
} else {
    $etat = "NORMAL";
}

echo "Charge CPU : " . $chargeCpu . " %<br>";
echo "État : " . $etat;
```

Testez au minimum les valeurs suivantes : `95`, `90`, `75`, `70`, `69` et `20`.



La même classification est disponible en version autonome dans `cours/sandbox-online/03-conditions.php` : copiez-collez-la sur https://3v4l.org pour tester les frontières sans Docker (modifiez simplement la valeur de `$chargeCpu`).

Complétez le tableau avant chaque exécution.

Charge CPU	État prévu	État obtenu
`95`	…	…
`90`	…	…
`75`	…	…
`70`	…	…
`69`	…	…
`20`	…	…

QUESTION: Pourquoi faut-il tester `>= 90` avant `>= 70` ?

9. Activité 5 - Combiner plusieurs conditions

9.1. Opérateurs logiques

Opérateur	Signification	Expression vraie lorsque…
`&&`	ET logique	les deux conditions sont vraies.
`||`	OU logique	au moins une condition est vraie.
`!`	NON logique	la condition qui suit est fausse.

9.2. Tester la disponibilité d’un service

Créez `disponibilite.php` :

```php
<?php

$serveurActif = true;
$serviceDemarre = true;

if ($serveurActif && $serviceDemarre) {
    echo "Le service est disponible.";
} else {
    echo "Le service est indisponible.";
}
```

Testez les quatre combinaisons possibles de `true` et `false`.

Serveur actif	Service démarré	Résultat attendu
`true`	`true`	Disponible
`true`	`false`	Indisponible
`false`	`true`	Indisponible
`false`	`false`	Indisponible

9.3. Déclencher une alerte sur plusieurs ressources

Créez `alerte-ressources.php`.

Une alerte critique doit être affichée si la charge CPU est supérieure ou égale à 90 % **ou** si la mémoire libre est strictement inférieure à 2 Go.

Données de départ :

```php
<?php

$chargeCpu = 50;
$memoireLibreGo = 1.5;
```

Écrivez la condition et testez au moins ces trois situations :

Charge CPU	Mémoire libre	Résultat attendu
`95`	`8`	Alerte critique
`50`	`1.5`	Alerte critique
`50`	`8`	Aucune alerte critique

QUESTION: Pourquoi faut-il utiliser `||` et non `&&` dans cette règle ?

10. Activité 6 - Déterminer l'état global d'un serveur

Créez `supervision.php` avec les variables suivantes :

```
<?php

$nomServeur = "srv-bdd-01";
$estActif = true;
$modeMaintenance = false;
$chargeCpu = 75;
$memoireLibreGo = 8;
```

Le programme doit déterminer `$etatGlobal` en respectant exactement l'ordre suivant :

1. si le serveur n'est pas actif : `HORS SERVICE` ;
2. sinon, s'il est en maintenance : `MAINTENANCE` ;
3. sinon, si la charge CPU atteint 90 % ou si la mémoire libre est inférieure à 2 Go : `CRITIQUE` ;
4. sinon, si la charge CPU atteint 70 % ou si la mémoire libre est inférieure à 4 Go : `SURVEILLANCE` ;
5. sinon : `NORMAL`.

Affichez ensuite le nom du serveur et son état global.

CONTRAINTE: Utilisez uniquement `if`, `elseif`, `else`, `&&`, `||` et `!`. N'utilisez ni opérateur ternaire, ni `match`.

Testez les six jeux de données suivants en modifiant les variables une situation à la fois.

Actif	Maintenance	CPU	Mémoire	État attendu
`false`	`false`	`40`	`8`	`HORS SERVICE`
`true`	`true`	`95`	`1`	`MAINTENANCE`
`true`	`false`	`95`	`8`	`CRITIQUE`
`true`	`false`	`50`	`1.5`	`CRITIQUE`
`true`	`false`	`75`	`8`	`SURVEILLANCE`
`true`	`false`	`40`	`8`	`NORMAL`

QUESTION: Pourquoi la situation « maintenance » doit-elle être testée avant les seuils de ressources ?

11. Activité 7 - Corriger des erreurs logiques

Ouvrez le fichier `diagnostic.php` fourni.

Le programme s'exécute, mais il ne respecte pas les règles suivantes :

1. le serveur est actif uniquement si `$etatServeur` est strictement égal à la chaîne `"actif"` ;
2. une charge CPU de 90 % est déjà critique ;
3. l'état des ressources est critique dès qu'au moins une des deux ressources dépasse son seuil ;
4. le message de maintenance est affiché uniquement lorsque `$modeMaintenance` vaut `true`.

Pour chaque erreur :

1. relevez la ligne concernée ;
2. expliquez l'erreur ;
3. corrigez uniquement l'opérateur fautif ;
4. vérifiez le résultat dans le navigateur.

Examinez ensuite seulement les modifications de ce fichier :

```
git diff -- 03-conditions/diagnostic.php
```

12. Exercice autonome - Autoriser une mise à jour

Créez `mise-a-jour.php` avec les variables suivantes :

```
<?php

$utilisateurAdministrateur = true;
$serveurActif = true;
$modeMaintenance = true;
$espaceLibreGo = 15;
```

Une mise à jour doit être refusée avec le premier message applicable :

1. `ACCÈS REFUSÉ` si l'utilisateur n'est pas administrateur ;
2. `SERVEUR ARRÊTÉ` si le serveur n'est pas actif ;
3. `MODE MAINTENANCE REQUIS` si le mode maintenance n'est pas activé ;
4. `ESPACE DISQUE INSUFFISANT` si l'espace libre est inférieur à 10 Go ;
5. `MISE À JOUR AUTORISÉE` dans tous les autres cas.

CONTRAINTES:

- utilisez une seule structure `if/elseif/else` ;
- respectez l'ordre des règles ;
- utilisez au moins une fois l'opérateur `!` ;

- n'utilisez ni ternaire, ni `match` ;
- testez les cinq résultats possibles.

13. Travail sur papier

Sans ordinateur, étudiez le programme suivant :

```
<?php

$serveurActif = true;
$chargeCpu = 92;
$memoireLibreGo = 3;

if (!$serveurActif) {
    $etat = "HORS SERVICE";
} elseif ($chargeCpu >= 90 || $memoireLibreGo < 2) {
    $etat = "CRITIQUE";
} elseif ($chargeCpu >= 70 || $memoireLibreGo < 4) {
    $etat = "SURVEILLANCE";
} else {
    $etat = "NORMAL";
}

echo $etat;
```

1. Donnez le résultat de chaque expression booléenne, dans l'ordre où elle est évaluée.
2. Indiquez la branche exécutée.
3. Écrivez exactement le texte affiché.
4. Donnez l'état obtenu avec `$chargeCpu = 65` et `$memoireLibreGo = 3`.
5. Remplacez `||` par `&&` dans le test critique. Quel serait alors le résultat avec les valeurs initiales ?

14. Exercices bonus

Facultatifs et gradués, dans le périmètre de la séance (comparaisons, `&&`, `||`, `!`, `if/elseif/else` ; ni ternaire, ni `match`, ni boucle, ni tableau). Autonomes : testez-les dans Docker ou sur https://3v4l.org.

- `[*]` accessible ;
- `[**]` intermédiaire ;
- `[*]` défi.

14.1. Exercice bonus 1 [*] - Fenêtre de maintenance

Les interventions de maintenance ne sont autorisées que la nuit : entre 22 h et 5 h inclus. Créez `fenetre-maintenance.php` :

```php
<?php

$heure = 3; // heure sur 24 h (0 à 23)
```

Écrivez une condition qui affiche `Maintenance autorisée (créneau de nuit)` si l'heure est comprise dans le créneau, et `Maintenance interdite en journée` sinon.

Testez au moins : `3` (autorisé), `23` (autorisé), `6` (interdit), `14` (interdit).

QUESTION: pourquoi faut-il un `||` et non un `&&` pour décrire ce créneau qui « passe minuit » ?

14.2. Exercice bonus 2 [**] - Éligibilité à une sauvegarde

Une sauvegarde automatique ne se déclenche que si **toutes** ces conditions sont réunies :

- le serveur est actif ;
- il n'est **pas** en maintenance ;
- l'espace libre est d'au moins 20 Go.

Données de départ :

```php
<?php

$serveurActif = true;
$modeMaintenance = false;
$espaceLibreGo = 25;
```

Écrivez la condition (avec `&&` et au moins un `!`) affichant `Sauvegarde lancée` ou `Sauvegarde reportée`. Testez les quatre situations suivantes :

Serveur actif	Maintenance	Espace libre	Résultat attendu
`true`	`false`	`25`	Sauvegarde lancée
`false`	`false`	`25`	Sauvegarde reportée
`true`	`true`	`25`	Sauvegarde reportée
`true`	`false`	`10`	Sauvegarde reportée

14.3. Exercice bonus 3 [*] - Défi : prévoir la sortie

Sans exécuter, prévoyez la sortie **exacte** de chaque ligne, puis vérifiez dans `logique.php` :

```php
<?php
```

```php
$a = 5;
$b = 10;

var_dump($a > 0 && $b > 0);
var_dump($a > 0 || $b > 100);
var_dump(!($a === 5));
var_dump($a !== "5");
var_dump($a > 0 && $b < 0 || $b === 10);
```

Pour la dernière ligne, précisez quel opérateur (`&&` ou `||`) est évalué en premier et pourquoi le résultat final est `true`.

15. Synthèse

Complétez les phrases.

1. Une condition produit la valeur … ou …
2. L'opérateur d'affectation est …
3. L'opérateur d'égalité stricte est …
4. L'opérateur « différent strictement » est …
5. L'opérateur logique ET est …
6. L'opérateur logique OU est …
7. L'opérateur logique NON est …
8. Une structure qui prévoit exactement deux chemins utilise … et …
9. Lorsque plusieurs seuils se recouvrent, il faut tester en premier …

16. Validation Git

Vérifiez les fichiers modifiés :

```
git status
```

Ajoutez uniquement le dossier de la séance :

```
git add 03-conditions
```

Créez le second commit de la séance puis publiez-le :

```
git commit -m "Ajoute la classification de l'état des serveurs"
git push
```

Vérifiez enfin :

```
git status
git log --oneline
```

Le dépôt doit être propre. Les deux commits de la séance doivent être visibles sur la forge.

17. Arrêter le serveur PHP

En fin de séance, dans le terminal (vos fichiers restent) :

```
docker compose down
```